

Incremental Development Plan and Final Report

Estimating Solar Eclipse Coverage and Sunspot Activity from Consumer Smart-Telescope Imagery
Using Verifiable, Non-Generative AI Methods

A lightweight computer-vision pipeline for the April 2024 total solar eclipse

The solar analysis that used to need an observatory now runs on a phone-sized telescope and a free notebook.

Author	Matthew
Instrument	Vaonis Hestia smart-telescope
Observation	Total solar eclipse, April 7 to 8, 2024, Smugglers Notch, Vermont
Dataset	50 telescope frames (images captured by Mrs. Michelle McCrohan)
Development window	June 15 to 23, 2026
Peer reviewers	Edgar and Aaron

Part 1. Project Overview

1.1 The problem and what I was trying to do

Measuring the Sun used to need real equipment: good optics, a dedicated observatory, and software that cost more than most people's cars. I wanted to find out how much of that I could do with a consumer smart-telescope and a notebook on any laptop. The telescope is a Vaonis Hestia that pairs with a phone, and the input is a folder of ordinary JPEGs it captured during the April 2024 total solar eclipse at Smugglers Notch, Vermont. From those images, my pipeline measures how much of the Sun the Moon is covering, counts sunspots on the visible disk, and assigns each frame a phase label.

I want to be honest about the scope. This is not a state-of-the-art benchmark. It is a checkable measurement of how a simple classical computer-vision pipeline does on real consumer data, with an honest account of where it works and where it doesn't. Every number comes straight from the pixels by a fixed procedure I can trace by hand.

1.2 What the pipeline does

The system handles three tasks, all with classical methods and no generative AI anywhere:

- **Coverage estimation.** How much of the solar disk is the Moon hiding in this frame, as a percentage, with a confidence flag.
- **Sunspot detection.** How many sunspots are on the visible photosphere, but only when enough of the disk is showing for that to mean anything.
- **Phase classification.** Is this frame a full disk, large crescent, moderate crescent, or thin crescent? This is the same classification problem NASA ran a coding competition around in 2024.

The methods are standard image processing: grayscale conversion, Otsu thresholding, morphological cleanup, connected-component analysis, RANSAC circle fitting for the solar limb, Gaussian background subtraction for sunspot detection, and a k-nearest-neighbors classifier for phase labeling. No neural network. Nothing generative. That was a deliberate choice and it is most of what makes the project worth doing.

1.3 The idea behind the project

There is a pattern in computing where something that once needed expensive, expert-only tools shows up later as a lightweight version anyone can run. Photo editing is the obvious example. The same thing has been happening with solar imaging. The disk photometry, sunspot tracking, and phase sorting that NASA and professional observatories built with large equipment can now be approximated on hardware a hobbyist already owns.

That is what this project is. I took established science and well-worn classical image operations and ran them on a consumer smart-telescope and a free VS Code notebook. It does not replace a professional instrument. The point is to make a useful slice of that analysis reachable to anyone.

1.4 Why it matters outside the classroom

Solar activity affects satellites, power grids, polar flight routes, and radio communications. Institutions already run professional flare prediction software. A lightweight pipeline like this one is a small step

toward that kind of monitoring being something a school or a hobbyist club can run on their own, for free.

1.5 How peer review shaped the work

Two classmates, Edgar and Aaron, reviewed my proposal and gave feedback I built into the plan.

- **Edgar** wanted a deployment platform that a class could actually run without setup headaches. He also asked me to test the pipeline on at least one image from a different source to show it generalizes beyond one site. And he pushed for the work to be framed against what professionals do, not just presented in isolation.
- **Aaron** focused on presentation. He asked me to map the pipeline stages visually so a non-technical person can follow what feeds into what, and to lead with the hook rather than bury it. He also pointed out that combining several non-generative methods into one working pipeline is itself the contribution, not just the accuracy number.

Both of them landed on the lightweight-tools framing as the thing holding the project together. The architecture diagram in Part 4 is my answer to Aaron's request. The held-out generalization test is my answer to Edgar's. VS Code with local Python is the deployment platform.

1.6 How this report is organized

The project ran in three increments from June 15 to June 23, 2026, each building on the last. Part 2 covers the proof of concept, where I built and tested the single-image core. Part 3 covers the functional prototype, where I scaled it to all 50 frames and trained the classifier. Part 4 covers the final product, where I added error analysis, a generalization test, and locked in the deployment approach. Part 5 puts all three side by side.

Part 2. Initial Increment: Proof of Concept

Development window: June 15 to 16, 2026. The goal was simple: show that one Hestia frame can produce a measurement I can actually trust.

2.1 Goals

- Load the dataset and understand the range of conditions across the 50 frames.
- Build the segmentation core that separates the lit Sun from the dark background and the telescope's own artifacts.
- Fit the solar limb, get a first coverage number, and check it on a full-disk frame where the answer should be close to zero.
- Confirm that sunspots show up on a clean full-disk frame.

2.2 What I did

On June 15 I pulled the reference contact times for the eclipse from NASA so I could check my results against the real timeline later. On June 16 I put together the single-image pipeline and tested it on frames I picked by hand.

The dataset is 50 frames across two days: April 7, the day before, which gave me clean full-disk practice images, and April 8, eclipse day. The frames vary a lot, bright white full disks, dim tan disks, off-center framing, thin crescents near totality. That variety matters because it is exactly what decides where the method works and where it doesn't.

The segmentation pipeline works in four steps on each frame: grayscale conversion with Gaussian blur to soften noise, Otsu thresholding to split the bright Sun from the dark background, morphological cleanup to drop specks and fill holes, and connected-component analysis to pick the right blob. Two things needed explicit handling that I hadn't planned for. The Hestia leaves a faint circular vignette ring in every frame, and a naive threshold picks it up as signal. I added a size-based rejection rule to throw it out. The camera also burns a watermark strip at the bottom of every frame, so I mask that row range before thresholding.

The calibration result was what convinced me the idea was sound. On a full-disk April 8 frame, the pipeline reported 0.18 percent coverage and found two sunspots. The expected coverage is essentially zero, so landing at 0.18 with no hand-tuning was a clean pass. The sunspot count also came out right without any nudging toward two. That was enough to build on.

2.3 What this increment taught me

The main lesson was that consumer telescope frames carry baggage that has nothing to do with the Sun, the vignette ring and the watermark strip, and you have to handle both explicitly or they corrupt every measurement. I also learned that robustness to exposure and framing has to be designed in from the start. The calibration result gave me confidence to scale up: if a full disk reads near zero and the sunspot count is right, the foundation is solid.

Part 3. Intermediate Increment: Functional Prototype

Development window: June 17 to 19, 2026. The single-image core had to scale to all 50 frames, produce the measurements spreadsheet, and feed a trained classifier.

3.1 Goals

- Run the pipeline over all 50 frames and write the results to one structured spreadsheet.
- Plot coverage against time and check that it follows the real physical arc of the eclipse.
- Run sunspot detection across the dataset and confirm the count holds steady on full-disk frames.
- Build the feature table, assign phase labels, and train a small classifier.

3.2 What I did

On June 17 I wrapped the single-image routine in a two-pass batch driver. Pass 1 segments and fits every frame and collects all the reliable fitted radii. The median of those becomes the calibrated fallback radius, R_0 , which is used in Pass 2 for any frame where the limb fit wasn't reliable enough to trust on its own. Pass 2 runs coverage measurement, sunspot detection, luminosity, and crescent feature extraction on all 50 frames using the calibrated R_0 as a backstop, then writes everything to the spreadsheet.

The spreadsheet is the central deliverable the proposal called for. Each row is one frame, with columns for timestamp, coverage percentage, fitted and calibrated radius, fit RMS error, confidence flag, sunspot count, relative luminosity, crescent angle, centroid offset, luminosity normalized to the full-disk median, and the phase label. Low-confidence fits are flagged so downstream plots and the classifier can treat them with appropriate skepticism.

When I plotted coverage against time for all 45 April 8 frames, the eclipse arc came through clearly: coverage starts near zero, climbs toward the maximum around 15:26, and drops back to zero as the Moon moves off the disk. That is the real test against physics, and the pipeline passed it across the full sequence, not just on hand-picked frames. The scatter in the middle phases was also informative. Several frames sit well off the smooth trend, and when I chased those down they came from off-center framing and exposure changes, not random noise. That is what drove the error analysis in Part 4.

Sunspot detection held up across the dataset. Every near-full-disk April 8 frame came back with two spots, matching the known solar activity that day. The detector is gated to only run when more than 55 percent of the disk is visible, which is why crescent frames correctly show zero rather than reporting noise.

3.3 The phase classifier

I trained a k-nearest-neighbors classifier ($k=5$) on four features: coverage percentage, centroid offset, crescent orientation angle, and normalized luminosity. The split was 35 training frames and 15 held-out test frames, stratified by phase. On the held-out set the classifier hit 93 percent accuracy. Logistic regression and a decision tree came in at 96 and 94 percent under cross-validation.

To check whether the features carry real information beyond coverage alone, I trained a version using only the morphological features with coverage removed. It still hit 86 percent under cross-validation.

The classifier is picking up on the actual shape of the eclipse, not just re-sorting a single number.

3.4 What this increment taught me

Running everything through one spreadsheet exposed problems that single-image testing hid completely. The scatter in the coverage curve was invisible until I had all 45 points plotted together, and seeing it is what forced the confidence flag and the error analysis later. The morphology-only classifier was a genuine surprise. I expected to need coverage to get decent accuracy. Getting 86 percent without it showed that the combined non-generative features have more signal than I had assumed.

Part 4. Final Product

Development window: June 20 to 23, 2026. This stretch was about cross-checks, a clear architecture diagram, a generalization test, honest error analysis, and locking in deployment.

4.1 Goals

- Draw the full pipeline as one diagram a non-technical person can follow.
- Evaluate the classifier with a confusion matrix on held-out data.
- Cross-check sunspot results and pin down where coverage error concentrates.
- Run the pipeline on a frame it had never seen, and settle the deployment platform.

4.2 What I did

June 20 went to cross-checking the sunspot results against the known April 8 solar activity and working out where coverage errors concentrate. June 21 was final plots and the architecture diagram. June 22 and 23 were this report and the presentation.

The architecture diagram shows the full pipeline on one page. A single Hestia frame enters and flows through grayscale conversion, thresholding, morphology, and component selection into the RANSAC limb fit. From the fit, four measurement branches run in parallel: coverage, sunspot detection, relative luminosity, and crescent features. Those four outputs combine into a feature vector the k-NN classifier turns into a phase label. The final outputs are the measurements spreadsheet, the coverage-versus-time curve, sunspot counts, and phase labels. Every box is a classical, deterministic operation. Nothing in the pipeline is a black box.

The confusion matrix on the held-out test set showed one miss: a full-disk frame classified as large crescent. That is a neighboring-phase slip, not a wild error. All other 14 test frames landed in the correct class. The mistakes land exactly where the proposal predicted they would, at adjacent phase boundaries.

The relative luminosity curve tracks brightness across the full sequence, normalized to the full-disk median. It does not depend on the circle fit at all, so it is an independent confirmation of the same arc the coverage curve shows. It also turned out to be a useful classifier feature because it falls smoothly even on frames where the limb fit is unreliable.

The error analysis pinned down where the method degrades. Circle-fit RMS error climbs as coverage increases toward totality, and the share of low-confidence frames spikes in the thin-crescent range. That is not a surprise: with only a thin sliver of the solar limb visible, RANSAC has less arc to fit and the result is less reliable. The failures are structured and predictable, which is what makes the confidence flag meaningful. The pipeline knows when not to trust itself.

The generalization test ran the pipeline on 115024.jpeg, a frame not used at any point during development. It returned a coverage reading of 2.3 percent, which is correct for a near-full disk. There was a catch: this particular frame is overexposed, blown out to pure white, so the sunspot contrast is gone and the detector correctly found zero spots. That exposure dependence of sunspot detection is worth knowing. A truly independent source like NASA imagery is the right next validation step.

4.3 Deployment

The pipeline runs as a VS Code notebook with local Python. To get started, open a terminal in VS Code and run:

```
pip install opencv-python numpy pandas openpyxl scikit-learn
```

Then open the notebook and set two path variables at the top: SRC pointing at the folder of eclipse JPEGs, and OUT pointing at where you want results saved. Run the single cell. The notebook processes every JPEG in the folder, saves eclipse_measurements.xlsx to the output path, and prints the classifier accuracy at the end. No internet connection needed. No cloud account. Works on any machine with Python installed.

4.4 What this increment taught me

Characterizing where the pipeline fails turned out to be as important as showing where it works. The error analysis converts the scatter from Part 3 into a precise, defensible statement about where the measurements can be trusted and where they can't. That is what makes it a credible tool rather than a demo. The generalization test taught me that a held-out frame reading correctly is encouraging, but a different instrument entirely is what I need to make a strong claim about generalization. The overexposed frame also made the exposure dependence of sunspot detection concrete in a way I had not expected.

Part 5. Increment Summary

The three increments side by side: what each set out to do, what got done, and the key lesson from each.

Increment	Window	Goals and work	Outcome and lesson
Initial: Proof of Concept	June 15 to 16	Know the 50-frame dataset. Build the single-image segmentation core. Fit the solar limb and calibrate coverage on a full disk. Detect sunspots on a clean frame.	Full disk reads 0.18% covered and two sunspots found. Lesson: the vignette ring and watermark have to be handled explicitly, and robustness to exposure and framing has to be built in early.
Intermediate: Functional Prototype	June 17 to 19	Run all 50 frames into the measurements spreadsheet. Plot coverage over time. Confirm sunspot count consistency. Build features and train the classifier.	Coverage follows the eclipse arc. Classifier hits 93% on held-out data, 86% from morphology alone. Lesson: seeing all results together exposes scatter that single-image testing hides.
Final: Final Product	June 20 to 23	Draw the full architecture. Confusion matrix on held-out data. Pin down thin-crescent error. Run held-out generalization test. Lock in VS Code + local Python deployment.	Errors stay at adjacent-phase boundaries and in thin-crescent range. Held-out frame reads correctly. Lesson: nailing down where it fails is what makes it credible.

Closing note

Three increments in, I have a working non-generative pipeline that measures eclipse coverage, counts sunspots, and classifies eclipse phase from consumer smart-telescope frames, running free on any laptop with Python. More importantly, it is honest about where it should not be trusted.

The next steps are clear. Validate against a genuinely independent image source to back up the generalization claim. Improve handling of thin crescents near totality. Keep the notebook easy to pick up, two path variables and one pip command, so a school or hobbyist club can actually use it. The underlying idea stays the same throughout: capability that once lived inside expensive professional tools can be rebuilt as something light and within reach of ordinary people.
